Absolutely. Since you've already implemented and experimented with Stages 1–2.6, let's now move into Stage 3 — YAML-Based Application Configuration.

This stage is especially important for you because we're going to connect the YAML concepts to your real Enterprise RAG settings.py design.

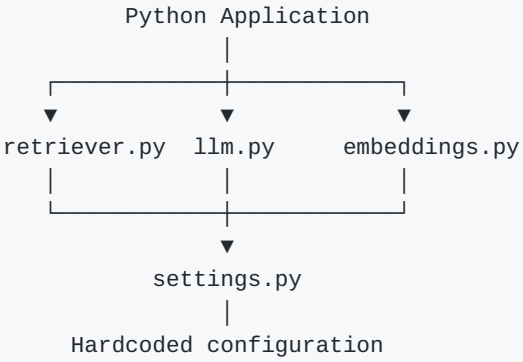
The key idea is:

YAML becomes the place where configuration values live, while settings.py becomes the clean Python interface that the rest of your RAG application uses.

Stage 3 — YAML-Based Application Configuration

3.1 First, understand the architecture

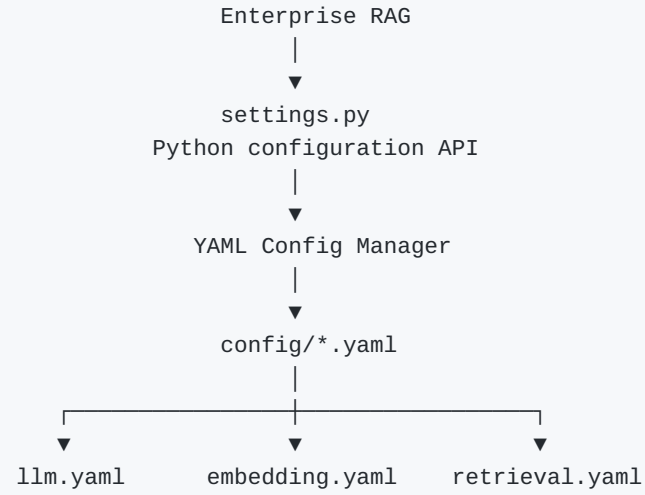
Your current Enterprise RAG probably has something like:



For example:

```
LLM_MODEL = "llama-3.1-8b-instant"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
CHROMA_DB_DIRECTORY = "./chroma_db"
```

We're going to change this to:



So your application code doesn't need to know that YAML exists.
That's an important architectural principle.

3.2 Our new project structure

Let's create a project specifically for this stage.
Your learning project should now look like:

```
yaml-learning/
│
├── config/
│   │
│   ├── app.yaml
│   ├── llm.yaml
│   ├── embedding.yaml
│   ├── vector_database.yaml
│   ├── retrieval.yaml
│   ├── chunking.yaml
│   └── logging.yaml
│
├── src/
│   │
│   ├── yaml_loader.py
│   ├── config_loader.py
│   └── settings.py
│
└── main.py
```

Notice that we're introducing two new files:

```
src/config_loader.py
src/settings.py
```

Their responsibilities are different.

3.3 Responsibility of each file

This distinction is extremely important.

yaml_loader.py

Responsible only for:

"How do I read one YAML file?"

```
YAML file
   ↓
load_yaml()
   ↓
Python dictionary
```

config_loader.py

Responsible for:

"How do I find and combine all my YAML configuration files?"

```
config/*.yaml
      ↓
load_all_configs()
      ↓
combined dictionary
```

settings.py

Responsible for:

"How should my Python application access configuration?"

```
combined configuration
        ↓
     Settings
        ↓
settings.llm.model
settings.embedding.model
settings.retrieval.top_k
```

main.py

The actual application.
It shouldn't care about:

- YAML
- PyYAML
- Path
- configuration directories

It should simply use:

```
settings.llm.model
```

That's the architecture we're building.

3.4 Step 1 — Create app.yaml

File:
`config/app.yaml`

Put:

```
application:
  name: Enterprise RAG application
  version: 1.0
  environment: development
```

3.5 Step 2 — Create llm.yaml

File:
`config/llm.yaml`

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: 0
```

3.6 Step 3 — Create embedding.yaml

File:
`config/embedding.yaml`

```
embedding:
  provider: sentence-transformers
  model: all-MiniLM-L6-v2
```

3.7 Step 4 — Create vector_database.yaml

File:
`config/vector_database.yaml`

```
vector_database:
  provider: chroma
  directory: ./chroma_db
  collection_name: enterprise_knowledge_base
```

3.8 Step 5 — Create retrieval.yaml

File:
`config/retrieval.yaml`

```
retrieval:
  top_k: 5
  reranking: false
```

3.9 Step 6 — Create chunking.yaml

File:
`config/chunking.yaml`

```
chunking:
  strategy: recursive
  chunk_size: 1000
  chunk_overlap: 200
```

This will become useful later when we connect this configuration to your RAG chunking implementation.

3.10 Step 7 — Create logging.yaml

File:
`config/logging.yaml`

```
logging:
  level: INFO
  file: logs/application.log
```

3.11 Step 8 — Create yaml_loader.py

We already created this during Stage 2.4.

File:
`src/yaml_loader.py`

Use:

```
from pathlib import Path
from typing import Any

import yaml


def load_yaml(file_path: str | Path) -> dict[str, Any]:
    """
    Read a YAML file and return its contents
    as a Python dictionary.
    """

    # Convert the incoming path into a Path object.
    file_path = Path(file_path)

    # Make sure the requested YAML file exists.
    if not file_path.exists():
        raise FileNotFoundError(
            f"YAML configuration file not found: {file_path}"
        )

    # Open the YAML file using UTF-8 encoding.
    with open(file_path, "r", encoding="utf-8") as file:

        # Convert YAML into a Python dictionary.
        config = yaml.safe_load(file)

    # Protect against an empty YAML file.
    if config is None:
        raise ValueError(
            f"YAML configuration file is empty: {file_path}"
        )

    return config
```

Important line

```
config = yaml.safe_load(file)
```

This is where:

```
YAML
 ↓
Python dictionary
```

actually happens.

3.12 Step 9 — Create config_loader.py

Now we're going to build the configuration manager.

File:
`src/config_loader.py`

```
from pathlib import Path
from typing import Any

from yaml_loader import load_yaml


def load_all_configs(config_dir: str | Path) -> dict[str, Any]:
    """
    Discover all YAML files in the configuration directory
    and merge them into one Python dictionary.
    """

    # Convert the directory path into a Path object.
    config_dir = Path(config_dir)

    # Make sure the configuration directory exists.
    if not config_dir.exists():
        raise FileNotFoundError(
            f"Configuration directory not found: {config_dir}"
        )

    # Start with an empty dictionary.
    configs = {}

    # Find every .yaml file in the configuration directory.
    for config_file in config_dir.glob("*.yaml"):

        # Load the individual YAML file.
        config = load_yaml(config_file)

        # Make sure each YAML file contains a dictionary.
        if not isinstance(config, dict):
            raise ValueError(
                f"Configuration must be a dictionary: {config_file}"
            )

        # Add the configuration to the combined dictionary.
```

```
        configs.update(config)

    # Make sure at least one YAML file was found.
    if not configs:
        raise ValueError(
            f"No configuration found in: {config_dir}"
        )

    return configs
```

Now we have:

```
config/*.yaml
      |
      ▼
load_all_configs()
      |
      ▼
one Python dictionary
```

3.13 Step 10 — Now comes the important part: settings.py

This is the file that will eventually replace the role of your current Enterprise RAG settings.py.

File:
`src/settings.py`

For now, let's keep it simple:

```
from pathlib import Path

from config_loader import load_all_configs


# Find the root directory of our project.
BASE_DIR = Path(__file__).resolve().parent.parent

# Location containing all YAML configuration files.
CONFIG_DIR = BASE_DIR / "config"


# Load and combine all YAML configuration files.
_CONFIG = load_all_configs(CONFIG_DIR)
```

At this point:

```
_CONFIG
```

contains:

```
{
    "application": {...},
    "llm": {...},
    "embedding": {...},
    "vector_database": {...},
```

```
    "retrieval": {...},
    "chunking": {...},
    "logging": {...}
}
```

3.14 Expose configuration through simple variables

Now add to settings.py:

```
# Application configuration
APPLICATION_NAME = _CONFIG["application"]["name"]
APPLICATION_VERSION = _CONFIG["application"]["version"]
ENVIRONMENT = _CONFIG["application"]["environment"]


# LLM configuration
LLM_PROVIDER = _CONFIG["llm"]["provider"]
LLM_MODEL = _CONFIG["llm"]["model"]
LLM_TEMPERATURE = _CONFIG["llm"]["temperature"]


# Embedding configuration
EMBEDDING_PROVIDER = _CONFIG["embedding"]["provider"]
EMBEDDING_MODEL = _CONFIG["embedding"]["model"]


# Vector database configuration
VECTOR_DB_PROVIDER = _CONFIG["vector_database"]["provider"]
CHROMA_DB_DIRECTORY = _CONFIG["vector_database"]["directory"]
COLLECTION_NAME = _CONFIG["vector_database"]["collection_name"]


# Retrieval configuration
TOP_K = _CONFIG["retrieval"]["top_k"]
RERANKING_ENABLED = _CONFIG["retrieval"]["reranking"]


# Chunking configuration
CHUNKING_STRATEGY = _CONFIG["chunking"]["strategy"]
CHUNK_SIZE = _CONFIG["chunking"]["chunk_size"]
CHUNK_OVERLAP = _CONFIG["chunking"]["chunk_overlap"]


# Logging configuration
LOG_LEVEL = _CONFIG["logging"]["level"]
LOG_FILE = _CONFIG["logging"]["file"]
```

Now something very interesting has happened.
Your YAML contains:

```
llm:
  model: llama-3.1-8b-instant
```

But your Python application can simply use:

```
LLM_MODEL
```

The application doesn't need to know about YAML.

3.15 Create main.py

Now create:

File:
`main.py`

Put:

```
from src.settings import (
    APPLICATION_NAME,
    LLM_MODEL,
    EMBEDDING_MODEL,
    COLLECTION_NAME,
    TOP_K,
    CHUNK_SIZE,
    CHUNK_OVERLAP,
)


print("Application:", APPLICATION_NAME)
print("LLM Model:", LLM_MODEL)
print("Embedding Model:", EMBEDDING_MODEL)
print("Collection:", COLLECTION_NAME)
print("Top K:", TOP_K)
print("Chunk Size:", CHUNK_SIZE)
print("Chunk Overlap:", CHUNK_OVERLAP)
```

Run from the project root:

```
python main.py
```

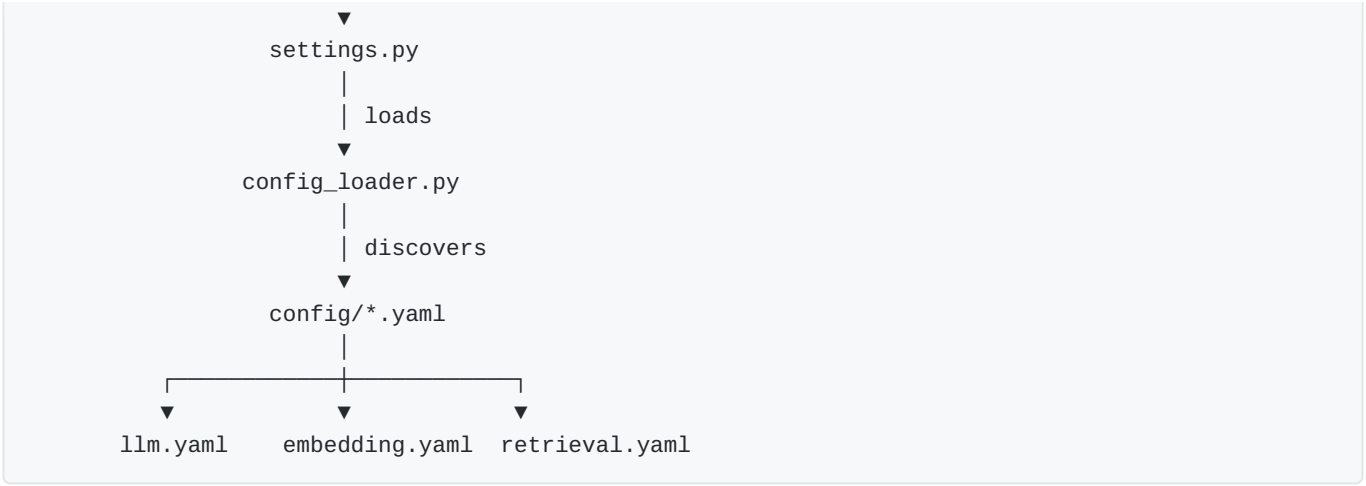
You should see:

```
Application: Enterprise RAG application
LLM Model: llama-3.1-8b-instant
Embedding Model: all-MiniLM-L6-v2
Collection: enterprise_knowledge_base
Top K: 5
Chunk Size: 1000
Chunk Overlap: 200
```

3.16 Look at what we've built

This is the important part.

```
                    main.py
                       |
                       | imports
```



And:

```
main.py
   │
   ├── LLM_MODEL
   ├── EMBEDDING_MODEL
   ├── COLLECTION_NAME
   └── TOP_K
```

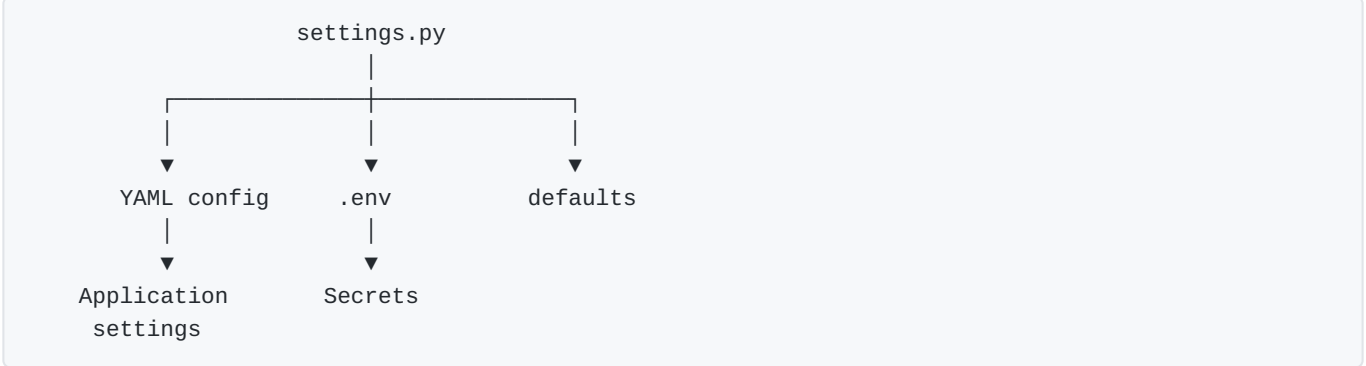
does not care that the values originated from YAML.
That's exactly what we wanted.

3.17 Now compare this with your real Enterprise RAG

Your original settings.py contained:

```
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

PDF_DIRECTORY = "./data/pdfs"

EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"

CHROMA_DB_DIRECTORY = "./chroma_db"

COLLECTION_NAME = "enterprise_knowledge_base"

LLM_MODEL = "llama-3.1-8b-instant"
```

Our new architecture could eventually become:



And this is important:

Do NOT put your GROQ_API_KEY into YAML.
Keep secrets such as:

- GROQ_API_KEY
- OPENAI_API_KEY
- AZURE_OPENAI_API_KEY

in .env, Azure Key Vault, or another secrets-management system.

YAML should contain things like:

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: 0
```

while .env contains:

```
GROQ_API_KEY=xxxxxxxx
```

That's a very common production pattern.

3.18 One important architectural improvement

You may be thinking:

"But now my settings.py still contains configuration variable definitions."
Correct — and that's intentional.
We're not trying to eliminate settings.py.
We're changing its responsibility.

Old settings.py

```
settings.py
     │
     └── stores configuration values
```

New settings.py

```
settings.py
     │
     └── exposes configuration to Python
               ▲
               │
        YAML provides values
```

Therefore your existing RAG files can continue doing:

```
from settings import LLM_MODEL
```

while the actual value lives in:

```
config/llm.yaml
```

This is why I described settings.py as the clean Python configuration interface.

3.19 Your first real Enterprise RAG migration

Eventually your old:

```
# settings.py

PDF_DIRECTORY = "./data/pdfs"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
CHROMA_DB_DIRECTORY = "./chroma_db"
COLLECTION_NAME = "enterprise_knowledge_base"
LLM_MODEL = "llama-3.1-8b-instant"
```

could become:

`config/app.yaml`

```
application:
  name: Enterprise RAG application
  version: 1.0

paths:
  pdf_directory: ./data/pdfs
```

`config/embedding.yaml`

```
embedding:
  provider: sentence-transformers
  model: sentence-transformers/all-MiniLM-L6-v2
```

`config/vector_database.yaml`

```
vector_database:
  provider: chroma
  directory: ./chroma_db
  collection_name: enterprise_knowledge_base
```

`config/llm.yaml`

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: 0
```

Then settings.py becomes the bridge between those files and your existing RAG code.

🎯 Stage 3 Exercise

Don't immediately modify your real Enterprise RAG.
Use your YAML learning project first.
Your task is to get this architecture working:

```
yaml-learning/
│
├── config/
│   ├── app.yaml
│   ├── llm.yaml
│   ├── embedding.yaml
│   ├── vector_database.yaml
│   ├── retrieval.yaml
│   ├── chunking.yaml
│   └── logging.yaml
│
├── src/
│   ├── yaml_loader.py
│   ├── config_loader.py
│   └── settings.py
│
└── main.py
```

And ultimately:

```
from src.settings import LLM_MODEL

print(LLM_MODEL)
```

should produce:

```
llama-3.1-8b-instant
```

Then change only:

```
# config/llm.yaml

model: some-new-model
```

Run:

```
python main.py
```

and verify that Python picks up the new model without changing main.py.

That experiment will make the entire concept click:

Code stays the same; configuration changes outside the code.

After this works, the next logical step is Stage 3.1 — improving settings.py with a proper Settings object, which will lead naturally into Pydantic validation, type safety, nested configuration, and eventually .env + YAML together.